
CH-3 Cascading Style Sheet & CSS 3

What is CSS?

- **CSS** stands for **Cascading Style Sheets**
- Styles define **how to display** HTML elements
- Styles were added to HTML 4.0 **to solve a problem**
- **External Style Sheets** can save a lot of work
- External Style Sheets are stored in CSS files
- .css is the extension of css files.

A CSS (cascading style sheet) file allows you to separate your web sites (X)HTML content from it's style. As always you use your (X)HTML file to arrange the content, but all of the presentation (fonts, colors, background, borders, text formatting, link effects & so on...) are accomplished within a CSS.

CSS Comment

Comments can be used to explain why you added certain selectors within your css file. So as to help others who may see your file, or to help you remember what you we're thinking at a later date. You can add comments that will be ignored by browsers in the following manner.

```
/* This is a comment */
```

You will note that it begins with a / (forward slash) and than an * (asterisks) then the comment, then the closing tag which is just backward from the opening tag * (asterisks) then the / (forward slash).

Css Syntax

The syntax for CSS is different than that of (X)HTML markup. Though it is not too confusing, once you take a look at it. It consists of only 3 parts.

selector { property: value }

To set several properties in a single rule requires the individual declarations to be separated by semicolons:

selector { property1: value1; property2: value2; property3: value3 }

The selector is the (X)HTML element that you want to style. The property is the actual property title, and the value is the style you apply to that property.

Each selector can have multiple properties, and each property within that selector can have independent values. The property and value are separated with a colon and contained within curly brackets. Multiple properties are separated by a semi colon. Multiple values within a property are separated by commas, and if an individual value contains more than one word you surround it with quotation marks. As shown below.

```
body {
  background: #eeeeee;
  font-family: "Trebuchet MS", Verdana, Arial, serif;
}
```

As you can see in the above code I have separated the color from the font-family with a semi-colon, separated the various fonts with commas and contained the "Trebuchet MS" within quotations marks. The final result sets the body color to light grey, and sets the font to ones that most users will have installed on their computer.

I have changed the way I layout my code, but you can arrange it in one line if you choose. I find that it is more readable if I spread each property to a separate line, with a 2 space indentation.

Combining Selectors

You can combine elements within one selector in the following fashion.

```
h1, h2, h3, h4, h5, h6 {
  color: #009900;
  font-family: Georgia, sans-serif;
}
```

As you can see in the above code, I have grouped all the header elements into one selector. Each one is separated by a comma. The final result of the above code sets all headers to green and to the specified font. If the user does not have the first font I declared it will go to another sans-serif font the user has installed on their computer.

CSS Selectors

CSS selectors allow you to select and manipulate HTML element(s).

CSS selectors are used to "find" (or select) HTML elements based on their id, classes, types, attributes, values of attributes and much more.

The element Selector

The element selector selects elements based on the element name.

You can select all <p> elements on a page like this: (all <p> elements will be center-aligned, with a red text color)

Example

```
p {  
  text-align: center;  
  color: red;  
}
```

The id Selector

The id selector uses the id attribute of an HTML tag to find the specific element.

An id should be unique within a page, so you should use the id selector when you want to find a single, unique element.

To find an element with a specific id, write a hash character, followed by the id of the element.

The style rule below will be applied to the HTML element with id="para1":

Example

```
#para1 {  
  text-align: center;  
  color: red;  
}
```

The class Selector

The class selector finds elements with the specific class.

The class selector uses the HTML class attribute.

To find elements with a specific class, write a period character, followed by the name of the class:

In the example below, all HTML elements with class="center" will be center-aligned:

Example

```
.center {
  text-align: center;
  color: red;
}
```

CSS - Pseudo Classes

CSS pseudo-classes are used to add special effects to some selectors. You do not need to use JavaScript or any other script to use those effects. A simple syntax of pseudo-classes is as follows –

selector:pseudo-class {property: value}

CSS classes can also be used with pseudo-classes –

selector.class:pseudo-class {property: value}

The most commonly used pseudo-classes are as follows –

Sr.No.	Value & Description
1	:link Use this class to add special style to an unvisited link.
2	:visited

Use this class to add special style to a visited link.

:hover

3

Use this class to add special style to an element when you mouse over it.

:active

4

Use this class to add special style to an active element.

:focus

5

Use this class to add special style to an element while the element has focus.

:first-child

6

Use this class to add special style to an element that is the first child of some other element.

:lang

7

Use this class to specify a language to use in a specified element.

While defining pseudo-classes in a <style>...</style> block, following points should be noted –

- a:hover MUST come after a:link and a:visited in the CSS definition in order to be effective.
- a:active MUST come after a:hover in the CSS definition in order to be effective.
- Pseudo-class names are not case-sensitive.
- Pseudo-class are different from CSS classes but they can be combined.

The :link pseudo-class

The following example demonstrates how to use the *:link* class to set the link color. Possible values could be any color name in any valid format.

```
<html>
<head>
  <style type = "text/css">
    a:link {color:#000000}
```

```
</style>
</head>

<body>
  <a href = "">Black Link</a>
</body>
</html>
```

The :visited pseudo-class

The following is the example which demonstrates how to use the *:visited* class to set the color of visited links. Possible values could be any color name in any valid format.

```
<html>
<head>
  <style type = "text/css">
    a:visited {color: #006600}
  </style>
</head>

<body>
  <a href = "">Click this link</a>
</body>
</html>
```

The :hover pseudo-class

The following example demonstrates how to use the *:hover* class to change the color of links when we bring a mouse pointer over that link. Possible values could be any color name in any valid format.

```
<html>
<head>
  <style type = "text/css">
    a:hover {color: #FFCC00}
  </style>
</head>

<body>
```

```
<a href = "">Bring Mouse Here</a>
</body>
</html>
```

The :active pseudo-class

The following example demonstrates how to use the *:active* class to change the color of active links. Possible values could be any color name in any valid format.

```
<html>
<head>
  <style type = "text/css">
    a:active {color: #FF00CC}
  </style>
</head>

<body>
  <a href = "">Click This Link</a>
</body>
</html>
```

The :focus pseudo-class

The following example demonstrates how to use the *:focus* class to change the color of focused links. Possible values could be any color name in any valid format.

```
<html>
<head>
  <style type = "text/css">
    a:focus {color: #0000FF}
  </style>
</head>

<body>
  <a href = "">Click this Link</a>
</body>
</html>
```

The :first-child pseudo-class

The *:first-child* pseudo-class matches a specified element that is the first child of another element and adds special style to that element that is the first child of some other element.

To make :first-child work in IE <!DOCTYPE> must be declared at the top of document.

For example, to indent the first paragraph of all <div> elements, you could use this definition –

```
<html>
  <head>
    <style type = "text/css">
      div > p:first-child {
        text-indent: 25px;
      }
    </style>
  </head>

  <body>

    <div>
      <p>First paragraph in div. This paragraph will be indented</p>
      <p>Second paragraph in div. This paragraph will not be indented</p>
    </div>
    <p>But it will not match the paragraph in this HTML:</p>

    <div>
      <h3>Heading</h3>
      <p>The first paragraph inside the div. This paragraph will not be
effected.</p>
    </div>

  </body>
</html>
```

The :lang pseudo-class

The language pseudo-class *:lang*, allows constructing selectors based on the language setting for specific tags.

This class is useful in documents that must appeal to multiple languages that have different conventions for certain language constructs. For example, the French language typically uses angle brackets (< and >) for quoting purposes, while the English language uses quote marks (' and ').

In a document that needs to address this difference, you can use the *:lang* pseudo-class to change the quote marks appropriately. The following code changes the <blockquote> tag appropriately for the language being used –

```
<html>
  <head>
    <style type = "text/css">

      /* Two levels of quotes for two languages*/
      :lang(en) { quotes: '""' '""' '""' '""'; }
      :lang(fr) { quotes: "<<" ">>" "<" ">"; }
    </style>
  </head>

  <body>
    <p>...<q lang = "fr">A quote in a paragraph</q>...</p>
  </body>
</html>
```

CSS - Pseudo Elements

CSS pseudo-elements are used to add special effects to some selectors. You do not need to use JavaScript or any other script to use those effects. A simple syntax of pseudo-element is as follows –

selector:pseudo-element {property: value}

CSS classes can also be used with pseudo-elements –

selector.class:pseudo-element {property: value}

The most commonly used pseudo-elements are as follows –

Sr.No.	Value & Description
1	:first-line Use this element to add special styles to the first line of the text in a selector.
2	:first-letter Use this element to add special style to the first letter of the text in a selector.
3	:before Use this element to insert some content before an element.
4	:after Use this element to insert some content after an element.

The :first-line pseudo-element

The following example demonstrates how to use the *:first-line* element to add special effects to the first line of elements in the document.

```
<html>
<head>
  <style type = "text/css">
    p:first-line { text-decoration: underline; }
    p.noline:first-line { text-decoration: none; }
  </style>
</head>

<body>
  <p class = "noline">
    This line would not have any underline because this belongs to nline
  class.
  </p>

  <p>
```

The first line of this paragraph will be underlined as defined in the CSS rule above. Rest of the lines in this paragraph will remain normal. This example shows how to use `:first-line` pseudo element to give effect to the first line of any HTML element.

```
</p>
</body>
</html>
```

The `:first-letter` pseudo-element

The following example demonstrates how to use the `:first-letter` element to add special effects to the first letter of elements in the document.

```
<html>
<head>
  <style type = "text/css">
    p:first-letter { font-size: 5em; }
    p.normal:first-letter { font-size: 10px; }
  </style>
</head>

<body>
  <p class = "normal">
    First character of this paragraph will be normal and will have font size 10
px;
  </p>

  <p>
    The first character of this paragraph will be 5em big as defined in the
    CSS rule above. Rest of the characters in this paragraph will remain
    normal. This example shows how to use :first-letter pseduo element
    to give effect to the first characters of any HTML element.
  </p>
</body>
</html>
```

The `:before` pseudo-element

The following example demonstrates how to use the `:before` element to add some content before any element.

The :after pseudo-element

The following example demonstrates how to use the *:after* element to add some content after any element.

```
<html>
  <head>
    <style type = "text/css">
      p:after {
        content: url(/images/bullet.gif)
      }
    </style>
  </head>

  <body>
    <p> This line will be succeeded by a bullet.</p>
    <p> This line will be succeeded by a bullet.</p>
    <p> This line will be succeeded by a bullet.</p>
  </body>
</html>
```

Types of Stylesheet

Three Ways to Insert CSS

There are three ways of inserting a style sheet:

- External style sheet
- Internal style sheet
- Inline style

External Style Sheet

- An external style sheet is ideal when the style is applied to many pages. With an external style sheet, you can change the look of an entire Web site by changing just one file.
- Each page must include a link to the style sheet with the <link> tag. The <link> tag goes inside the head section:

```
<head>
<link rel="stylesheet" type="text/css" href="mystyle.css">
</head>
```

- An external style sheet can be written in any text editor. The file should not contain any html tags. The style sheet file must be saved with a .css extension. An example of a style sheet file is shown below:

```
"myStyle.css":
hr {color: sienna;}
p {margin-left: 20px;}
body {background-image: url("images/background.gif");}
```

Internal Style Sheet (Embedded)

An internal style sheet should be used when a single document has a unique style. You define internal styles in the head section of an HTML page, inside the <style> tag, like this:

```
<head>
<style>
hr {color: sienna;}
p {margin-left: 20px;}
body {background-image: url("images/background.gif");}
</style>
</head>
```

Inline Styles

An inline style loses many of the advantages of a style sheet (by mixing content with presentation). Use this method sparingly!

To use inline styles, add the style attribute to the relevant tag. The style attribute can contain any CSS property. The example shows how to change the color and the left margin of a paragraph:

```
<p style="color:sienna;margin-left:20px;">This is a paragraph.</p>
```

Multiple Style Sheets

If some properties have been set for the same selector in different style sheets, the values will be inherited from the more specific style sheet.

For example, assume that an external style sheet has the following properties for the h3 selector:

```
h3 {  
    color: red;  
    text-align: left;  
    font-size: 8pt;  
}
```

then, assume that an internal style sheet also has the following properties for the h3 selector:

```
h3 {  
    text-align: right;  
    font-size: 20pt;  
}
```

If the page with the internal style sheet also links to the external style sheet the properties for the h3 element will be:

```
color: red;  
text-align: right;  
font-size: 20pt;
```

The color is inherited from the external style sheet and the text-alignment and the font-size is replaced by the internal style sheet.

Multiple Styles Will Cascade into One

Styles can be specified:

- inside an HTML element
- inside the head section of an HTML page
- in an external CSS file

Cascading order

What style will be used when there is more than one style specified for an HTML element?

Generally speaking we can say that all the styles will "cascade" into a new "virtual" style sheet by the following rules, where number four has the highest priority:

1. Browser default
2. External style sheet
3. Internal style sheet (in the head section)
4. Inline style (inside an HTML element)

So, an inline style (inside an HTML element) has the highest priority, which means that it will override a style defined inside the <head> tag, or in an external style sheet, or in a browser (a default value).

CSS Margin

The CSS margin properties define the space around elements.

All CSS Margin Properties

Property	Description
<u>margin</u>	A shorthand property for setting the margin properties in one declaration
<u>margin-bottom</u>	Sets the bottom margin of an element
<u>margin-left</u>	Sets the left margin of an element
<u>margin-right</u>	Sets the right margin of an element
<u>margin-top</u>	Sets the top margin of an element

Margin

The margin clears an area around an element (outside the border). The margin does not have a background color, and is completely transparent.

The top, right, bottom, and left margin can be changed independently using separate properties. A shorthand margin property can also be used, to change all margins at once.

Possible Values

Value	Description
Auto	The browser calculates a margin
<i>Length</i>	Specifies a margin in px, pt, cm, etc. Default value is 0px
% element	Specifies a margin in percent of the width of the containing element
Inherit element	Specifies that the margin should be inherited from the parent element

Margin - Individual sides

In CSS, it is possible to specify different margins for different sides of an element:

Example

```
p {
  margin-top: 100px;  margin-bottom: 100px;
  margin-right: 150px; margin-left : 50px; }
```

Margin - Shorthand property

To shorten the code, it is possible to specify all the margin properties in one property. This is called a shorthand property.

The shorthand property for all the margin properties is "margin":

Example

```
p {
  margin: 100px 50px;
}
```

The margin property can have from one to four values.

- **margin: 25px 50px 75px 100px;**
 - top margin is 25px
 - right margin is 50px
 - bottom margin is 75px
 - left margin is 100px
- **margin: 25px 50px 75px;**
 - top margin is 25px
 - right and left margins are 50px
 - bottom margin is 75px
- **margin: 25px 50px;**
 - top and bottom margins are 25px
 - right and left margins are 50px
- **margin: 25px;**
 - all four margins are 25px

CSS Padding

The CSS padding properties define the space between the element border and the element content.

All CSS Padding Properties

Property	Description
<u>padding</u>	A shorthand property for setting all the padding properties in one declaration
<u>padding-bottom</u>	Sets the bottom padding of an element
<u>padding-left</u>	Sets the left padding of an element
<u>padding-right</u>	Sets the right padding of an element
<u>padding-top</u>	Sets the top padding of an element

Padding

The padding clears an area around the content (inside the border) of an element. The padding is affected by the background color of the element.

The top, right, bottom, and left padding can be changed independently using separate properties. A shorthand padding property can also be used, to change all paddings at once.

Possible Values

Value	Description
<i>Length</i>	Defines a fixed padding (in pixels, pt, em, etc.)
<i>%</i>	Defines a padding in % of the containing element

Padding - Individual sides

In CSS, it is possible to specify different padding for different sides:

Example

```
p {  
  padding-top: 25px;  
  padding-bottom: 25px;  
  padding-right: 50px;  
  padding-left: 50px;  
}
```

Padding - Shorthand property

To shorten the code, it is possible to specify all the padding properties in one property. This is called a shorthand property.

The shorthand property for all the padding properties is "padding":

Example

```
p { padding: 25px 50px; }
```

The padding property can have from one to four values.

- **padding: 25px 50px 75px 100px;**
 - top padding is 25px
 - right padding is 50px
 - bottom padding is 75px
 - left padding is 100px
- **padding: 25px 50px 75px;**
 - top padding is 25px
 - right and left paddings are 50px
 - bottom padding is 75px
- **padding: 25px 50px;**
 - top and bottom paddings are 25px
 - right and left paddings are 50px
- **padding: 25px;**
 - all four paddings are 25px

CSS Border

The CSS border properties allow you to specify the style and color of an element's border.

All CSS Border Properties

Property	Description
<u>border</u>	Sets all the border properties in one declaration
<u>border-bottom</u>	Sets all the bottom border properties in one declaration
<u>border-bottom-color</u>	Sets the color of the bottom border
<u>border-bottom-style</u>	Sets the style of the bottom border
<u>border-bottom-width</u>	Sets the width of the bottom border
<u>border-color</u>	Sets the color of the four borders

<u>border-left</u>	Sets all the left border properties in one declaration
<u>border-left-color</u>	Sets the color of the left border
<u>border-left-style</u>	Sets the style of the left border
<u>border-left-width</u>	Sets the width of the left border
<u>border-right</u>	Sets all the right border properties in one declaration
<u>border-right-color</u>	Sets the color of the right border
<u>border-right-style</u>	Sets the style of the right border
<u>border-right-width</u>	Sets the width of the right border
<u>border-style</u>	Sets the style of the four borders
<u>border-top</u>	Sets all the top border properties in one declaration
<u>border-top-color</u>	Sets the color of the top border
<u>border-top-style</u>	Sets the style of the top border
<u>border-top-width</u>	Sets the width of the top border
<u>border-width</u>	Sets the width of the four borders

Border Style

The border-style property specifies what kind of border to display.

border-style values:

none: Defines no border

dotted: Defines a dotted border

dashed: Defines a dashed border

solid: Defines a solid border

double: Defines two borders. The width of the two borders are the same as the border-width value

groove: Defines a 3D grooved border. The effect depends on the border-color value

ridge: Defines a 3D ridged border. The effect depends on the border-color value

inset: Defines a 3D inset border. The effect depends on the border-color value

outset: Defines a 3D outset border. The effect depends on the border-color value

Border Width

The border-width property is used to set the width of the border.

The width is set in pixels, or by using one of the three pre-defined values: thin, medium, or thick.

Example

```
p.one { border-style: solid; border-width: 5px; }  
p.two { border-style: solid; border-width: medium; }
```

Border Color

The border-color property is used to set the color of the border. The color can be set by:

- name - specify a color name, like "red"
- RGB - specify a RGB value, like "rgb(255,0,0)"
- Hex - specify a hex value, like "#ff0000"

You can also set the border color to "transparent".

Example

```
p.one { border-style: solid; border-color: red; }  
p.two { border-style: solid; border-color: #98bf21; }
```

Border - Individual sides

In CSS it is possible to specify different borders for different sides:

Example

```
p { border-top-style: dotted;  
border-right-style: solid;  
border-bottom-style: dotted;  
border-left-style: solid; }
```

Border - Shorthand property

As you can see from the examples above, there are many properties to consider when dealing with borders. To shorten the code, it is also possible to specify all the individual border properties in one property. This is called a shorthand property.

The border property is a shorthand for the following individual border properties:

- border-width
- border-style (required)
- border-color

Example: `p { border: 5px solid red; }`

CSS Text

This text is styled with some of the text formatting properties.

All CSS Text Properties

Property	Description
<u>color</u>	Sets the color of text
<u>direction</u>	Specifies the text direction/writing direction
<u>letter-spacing</u>	Increases or decreases the space between characters in a text
<u>line-height</u>	Sets the line height
<u>text-align</u>	Specifies the horizontal alignment of text
<u>text-decoration</u>	Specifies the decoration added to text
<u>text-indent</u>	Specifies the indentation of the first line in a text-block
<u>text-shadow</u>	Specifies the shadow effect added to text
<u>text-transform</u>	Controls the capitalization of text
<u>vertical-align</u>	Sets the vertical alignment of an element
<u>white-space</u>	Specifies how white-space inside an element is handled
<u>word-spacing</u>	Increases or decreases the space between words in a text

Text Color

The color property is used to set the color of the text.

With CSS, a color is most often specified by:

- a HEX value - like "#ff0000"
- an RGB value - like "rgb(255,0,0)"
- a color name - like "red"

The default color for a page is defined in the body selector.

Example

```
body { color: blue; }  
h1 { color: #00ff00; }
```

Text Alignment

The text-align property is used to set the horizontal alignment of a text.

Text can be centered, or aligned to the left or right, or justified.

text-align: left | right | center | justify

When text-align is set to "justify", each line is stretched so that every line has equal width, and the left and right margins are straight (like in magazines and newspapers).

Example

```
h1 { text-align: center; }  
p.date { text-align: right; }  
p.main { text-align: justify; }
```

Text Decoration

The text-decoration property is used to set or remove decorations from text.

text-decoration: none | [underline || overline || line-through || blink]

The text-decoration property is mostly used to remove underlines from links for design purposes:

Example

```
a { text-decoration: none; }
```

The text-decoration property defines how text is decorated. One or more settings can be made. For example:

```
h3 { text-decoration: underline blink }
```

Text Transformation

The text-transform property is used to specify uppercase and lowercase letters in a text.

It can be used to turn everything into uppercase or lowercase letters, or capitalize the first letter of each word.

```
text-transform: capitalize | uppercase | lowercase | none
```

The text-transform property allows text to be transformed by one of four properties:

Examples:

```
h1 { text-transform: uppercase }
```

```
h2 { text-transform: capitalize }
```

Text Indentation (set distance from left margin)

The text-indent property is used to specify the indentation of the first line of a text.

```
text-indent: <length> | <percentage>
```

Example

```
p { text-indent: 50px; }
```

Word Spacing

word-spacing: normal | <length>

The word-spacing property defines additional spacing between words. Negative values are permitted which allows the letters to be closed up or even overlap.

Example

```
p { word-spacing: 0.4em }
h1 { word-spacing: -0.2em }
h2 { word-spacing: normal }
```

Letter Spacing

letter-spacing: normal | <length>

The letter-spacing property defines additional spacing between characters. Negative values are permitted. A value of normal will still allow justification to take place. A setting of 0 is defined as preventing justification.

Example

```
h {letter-spacing: 0.2em }
```

Line Height

line-height: normal | <number> | <length> | <percentage>

The line-height property defines the spacing between baselines of two adjacent lines of text. It can be defined in points, inches, centimetres, or even pixels. When the value is a number, the line height is calculated by multiplying the element's font size by the number. Percentage values are relative to the element's font size. Negative values are not allowed.

The line-height property could be used to double space text:

p { line-height: 200% }

If the font-size is 10pt, the following rules all have the same effect:

p { line-height: 1.2 }

P { line-height: 1.2em }

P { line-height: 120% }

A value of normal is usually set to between 1.0 and 1.2

Vertical Alignment

vertical-align: baseline | sub | super | top | text-top | middle | bottom | text-bottom | <percentage>

The vertical-align property defines the vertical positioning of an inline element (such as some text or even an image) relative to the current baseline. It could be used to produce x^2 . The letter x in this case would be called the **parent** and the power 2 would be called the **child**.

The value may be a percentage of the element's line-height property. It specifies how much the element's baseline is raised above the parent's baseline. Negative values are permitted.

The other values are given in the following table.

Notation	Meaning
baseline	aligns baselines of element and with the parent's baseline
middle	aligns vertical midpoint of element with parent's
baseline plus half the x-height	(height of letter "x") of the parent
sub	subscripts the element
super	superscripts the element
text-top	aligns tops of element and top of parent element's font
text-bottom	aligns bottoms of element and bottom of parent
element's font	
top	aligns top of element with tallest element on the line
bottom	aligns bottom of element with lowest element on the line

Each font will have well-defined values for the aspects of the font defined in Figure 6.2 to aid it in achieving the various effects required for vertical alignment.

The vertical-align property is also useful for aligning images.

Example

img { vertical-align: middle }

text-shadow

The text-shadow property applies shadow to text.

text-shadow: *h-shadow v-shadow blur color*|none|initial|inherit;

Property Values

Value	Description
<i>h-shadow</i>	Required. The position of the horizontal shadow. Negative values are allowed
<i>v-shadow</i>	Required. The position of the vertical shadow. Negative values are allowed
<i>Blur</i>	Optional. The blur distance
<i>Color</i>	Optional. The color of the shadow.
None	Default value. No shadow
Initial	Sets this property to its default value.
Inherit	Inherits this property from its parent element.

CSS Color and Background

Color

color: <color>

The color property sets the foreground colour of a text element.

Example

```
h1 {color: blue }
h2 {color: rgb(255,0,0) }
```

Background

CSS background properties are used to define the background effects of an element.

CSS properties used for background effects:

- background-color
- background-image
- background-repeat
- background-attachment
- background-position

All CSS Background Properties

Property	Description
<u>background</u>	Sets all the background properties in one declaration
<u>background-attachment</u>	Sets whether a background image is fixed or scrolls with the rest of the page
<u>background-color</u>	Sets the background color of an element
<u>background-image</u>	Sets the background image for an element
<u>background-position</u>	Sets the starting position of a background image

[background-repeat](#) Sets how a background image will be repeated

Background Color

[background-color: <color> | transparent](#)

The background-color property specifies the background color of an element.

The background color of a page is defined in the body selector:

Example

```
body { background-color: #b0c4de; }
```

The value **transparent** indicates that whatever is behind the element can be seen. For example if the background to the **body** element was specified as being red, a block-level element with **backgroundcolor** set to **transparent** would appear with a red background.

Background Image

[background-image: <url> | none](#)

The background-image property specifies an image to use as the background of an element.

By default, the image is repeated so it covers the entire element.

The background image for a page can be set like this:

Example

```
body { background-image: url("paper.gif"); }
```

Background Repeat

[background-repeat: repeat | repeat-x | repeat-y | no-repeat](#)

If the background image does not fill the whole element area, this description specifies how it is repeated. The **repeat-x** value will repeat the image horizontally while the **repeat-y** value will repeat the image vertically. Setting **background-repeat** to **repeat** will repeat the image in both x and y directions.

Example

```
body { background-image: url(/images/monkey.gif) }
body { background-repeat: repeat-x }
```

Background Attachment

background-attachment: scroll | fixed

The background-image may be pasted on to the screen of the display in which case, when the elements that it is associated with move, they will be on top of a different part of the image. The alternative is that the background image is attached to the element, in which case, when the element moves, the background stays the same. The background-attachment value of scroll indicates that the background moves with the content, while fixed indicates it is fixed. For example, the following specifies a fixed background image:

```
body { background-image: url("monkey.jpg") }
body { background-attachment: fixed }
```

This tends to be mainly useful if the background image is attached to the **body** element. If the page is very long and the user has to scroll down the page to see the bottom, the background will remain fixed if **background-attachment** is set to **fixed**. For most other uses, the default value of **scroll** makes more sense.

Background Position

background-position: [percentage > | length]{1,2} | [top | center | bottom] || [left | center | right]

The effect of the background-image (especially when it is repeated) will depend significantly on the origin of the image relative to the display (in **fixed** attachment) and relative to the element (in **scroll** attachment).

The background-position property gives the initial position of the background image relative to the display or element depending on the attachment mode.

The easiest way to assign a background position is with the keywords:

Horizontal: left, center, right

Vertical : top, center, bottom

Background (set background images or color)

`background: [ <background-color> || <background-image> || <background-repeat> ||<background-attachment> || <background-position> ]`

The background property is a shorthand for the background-related properties.

```
body { background: white url(http://www.clrc/xyz.gif) }
p { background: url(../backgrounds/lion.png) #F0F000 fixed }
h1 { background: #0F0 url(grass.jpg) no-repeat bottom left }
ul { background: white url('monkey.jpg') repeat scroll 0% 50% }
```

CSS Font

CSS font properties define the font family, boldness, size, and the style of a text.

All CSS Font Properties

Property	Description
<u>font</u>	Sets all the font properties in one declaration
<u>font-family</u>	Specifies the font family for text
<u>font-size</u>	Specifies the font size of text
<u>font-style</u>	Specifies the font style for text

font-variant Specifies whether or not a text should be displayed in a small-caps font

font-weight Specifies the weight of a font

CSS Font Families(set Fontface)

font-family: [<family-name> | <generic-family>] [, [<family-name> | <generic-family>]]*

<family-name> ::= serif | sans-serif | cursive | fantasy | monospace

In CSS, there are two types of font family names:

- **generic family** - a group of font families with a similar look (like "Serif" or "Monospace")
- **font family** - a specific font family (like "Times New Roman" or "Arial")

Generic family	Font family	Description
Serif	Times New Roman Georgia	Serif fonts have small lines at the ends on some characters
Sans-serif	Arial Verdana	"Sans" means without - these fonts do not have the lines at the ends of characters
Monospace	Courier New Lucida Console	All monospace characters have the same width

Font Family

The font family of a text is set with the font-family property.

The font-family property should hold several font names as a "fallback" system. If the browser does not support the first font, it tries the next font.

Start with the font you want, and end with a generic family, to let the browser pick a similar font in the generic family, if no other fonts are available.

```
p { font-family: "Times New Roman", Times, serif; }
```

Font Style (Italicise text)

font-style: normal | italic | oblique

The font-style property is mostly used to specify italic text.

This property has three values:

- normal - The text is shown normally
- italic - The text is shown in italics
- oblique - The text is "leaning" (oblique is very similar to italic, but less supported)

Example

```
p.normal { font-style: normal; }
p.italic { font-style: italic; }
p.oblique { font-style: oblique; }
```

Font Variant (size of upper case)

font-variant: normal | small-caps

Normally, upper case characters are much larger than the lower-case characters. Small-caps are often used when all the letters of the word are in capitals. In this case the upper case characters are only slightly larger than the lowercase ones.

Example

p { font-variant: small-caps }

Font Weight (set thickness of type)

font-weight: normal | bold | bolder | lighter | 100 | 200 | 300 | 400 | 500 | 600 | 700 | 800 | 900

The font-weight property sets the weight of the font to values like **normal** and **bold**. It also has values **bolder** and **lighter** which are relative to any inherited font -weight. Block-level elements are displayed with default values inherited

from the **body** or **div** element that surrounds it. It is also possible to define absolute font-weights. Values range from 100 to 900 with **normal** being 400 and bold being **700** approximately. If that much control is not possible, some of the weights may be grouped together (for example, 100, 200 and 300 may all look the same) and if the specified weight is not available, an alternative value will be chosen.

Example

```
h1 { font-weight: 800 }
h2 { font-weight: bold }
h3 { font-weight: normal }
```

Font Size

font-size: <absolute-size> | <relative-size> | <length> | <percentage>

<absolute-size> ::= xx-small | x-small | small | medium | large | x-large | xx-large

<relative-size> ::= larger | smaller

The font-size property sets the size of the text.

Being able to manage the text size is important in web design. However, you should not use font size adjustments to make paragraphs look like headings, or headings look like paragraphs.

Always use the proper HTML tags, like <h1> - <h6> for headings and <p> for paragraphs.

The font-size value can be an absolute, or relative size.

Absolute size:

- Sets the text to a specified size
- Does not allow a user to change the text size in all browsers (bad for accessibility reasons)
- Absolute size is useful when the physical size of the output is known

Relative size:

- Sets the size relative to surrounding elements
- Allows a user to change the text size in browsers

Example

```
h1 { font-size: 40px; }
```

Font (Shorthand)

font: [<font-style> || <font-variant> ||<font-weight>]? || <font-size> [/<line-height>]? || <font-family>]

The font property may be used as a shorthand for the various font properties, as well as the line-height.

Example

```
p { font: italic bold 12pt/14pt Times, serif }
```

CSS Links

Links can be styled in different ways.

Styling Links

Links can be styled with any CSS property (e.g. color, font-family, background, etc.).

In addition, links can be styled differently depending on what **state** they are in.

The four links states are:

- a:link - a normal, unvisited link
- a:visited - a link the user has visited
- a:hover - a link when the user mouses over it
- a:active - a link the moment it is clicked

Example

```
/* unvisited link */ a:link { color: #FF0000; }  
/* visited link */ a:visited { color: #00FF00; }  
/* mouse over link */ a:hover { color: #FF00FF; }  
/* selected link */ a:active { color: #0000FF; }
```

When setting the style for several link states, there are some order rules:

- a:hover MUST come after a:link and a:visited
- a:active MUST come after a:hover

Common Link Styles

In the example above the link changes color depending on what state it is in.

Text Decoration

The text-decoration property is mostly used to remove underlines from links:

Example

```
a:link { text-decoration: none; }  
a:visited { text-decoration: none; }  
a:hover { text-decoration: underline; }  
a:active { text-decoration: underline; }
```

CSS Lists

The CSS list properties allow you to:

- Set different list item markers for ordered lists
- Set different list item markers for unordered lists
- Set an image as the list item marker

List

In HTML, there are two types of lists:

- unordered lists - the list items are marked with bullets
- ordered lists - the list items are marked with numbers or letters

With CSS, lists can be styled further, and images can be used as the list item marker.

All CSS List Properties

Property	Description
<u>list-style</u>	Sets all the properties for a list in one declaration
<u>list-style-image</u>	Specifies an image as the list-item marker
<u>list-style-position</u>	Specifies if the list-item markers should appear inside or outside the content flow
<u>list-style-type</u>	Specifies the type of list-item marker

List Style Type

`list-style-type: disc | circle | square | decimal | lower-roman | upper-roman | lower-alpha | upper-alpha | none`

The **list-style-type** property specifies the type of list-item marker to be used for ordered and unordered lists, and is used if: list-style-image is none; image loading is turned off; the image pointed to by the URL cannot be displayed.

Example

```
li { list-style-type: square }
ul { list-style-type: decimal }
ol { list-style-type: lower-alpha }
```

List Style Image

`list-style-image: <url> | none`

The list-style-image property defines the image to be used as the list-item marker when image loading is turned on. It replaces the marker specified in the list-style-type property.

Example

```
ul { list-style-image: url("arrow.gif") }  
li { list-style-image: url("triangle.png") }
```

List Style Position

list-style-position: inside | outside

This property defines where the marker is displayed relative to the list item. For **inside**, the lines will wrap under the marker instead of being indented while for **outside** it will not.

Example

```
ul { list-style: outside }  
ol { list-style: inside }
```

List Style

list-style: [<list-style-type> || <list-style-position> || <list-style-image>]

The list-style property is a shorthand for the list-style-type, list-style-position, and list-style-image properties.

Example

```
li { list-style: square inside }  
ul { list-style: circle url(triangle.png) outside }  
ol { list-style: upper-alpha none }
```

CSS Tables

Table Borders

To specify table borders in CSS, use the border property.

The example below specifies a black border for table, th, and td elements:

Example

```
table, th, td { border: 1px solid black; }
```

Notice that the table in the example above has double borders. This is because both the table and the th/td elements have separate borders.

To display a single border for the table, use the border-collapse property.

Collapse Borders

The border-collapse property sets whether the table borders are collapsed into a single border or separated:

Example

```
table { border-collapse: collapse; }  
table, th, td { border: 1px solid black; }
```

Table Width and Height

Width and height of a table is defined by the width and height properties.

The example below sets the width of the table to 100%, and the height of the th elements to 50px:

Example

```
table { width: 100%; }  
th { height: 50px; }
```

Table Text Alignment

The text in a table is aligned with the text-align and vertical-align properties.

The text-align property sets the horizontal alignment, like left, right, or center:

Example

```
td { text-align: right; }
```

The vertical-align property sets the vertical alignment, like top, bottom, or middle:

Example

```
td { height: 50px; vertical-align: bottom; }
```

Table Padding

To control the space between the border and content in a table, use the padding property on td and th elements:

Example

```
td { padding: 15px; }
```

Table Color

The example below specifies the color of the borders, and the text and background color of th elements:

Example

```
table, td, th { border: 1px solid green; }  
th { background-color: green; color: white; }
```

CSS Positioning**Positioning**

The CSS positioning properties allow you to position an element. It can also place an element behind another, and specify what should happen when an element's content is too big.

Elements can be positioned using the top, bottom, left, and right properties. However, these properties will not work unless the position property is set first. They also work differently depending on the positioning method.

position: static | relative | absolute | fixed | inherit

There are four different positioning methods.

Static Positioning

HTML elements are positioned static by default. A static positioned element is always positioned according to the normal flow of the page.

Static positioned elements are not affected by the top, bottom, left, and right properties.

Fixed Positioning

An element with fixed position is positioned relative to the browser window.

It will not move even if the window is scrolled:

Example

```
p.pos_fixed {  
  position: fixed;  
  top: 30px;  
  right: 5px;  
}
```

Fixed positioned elements are removed from the normal flow. The document and other elements behave like the fixed positioned element does not exist.

Fixed positioned elements can overlap other elements.

Relative Positioning

A relative positioned element is positioned relative to its normal position.

Example

```
h2.pos_left {  
  position: relative;  
  left: -20px;  
}  
h2.pos_right {  
  position: relative;  
  left: 20px;  
}
```

The content of relatively positioned elements can be moved and overlap other elements, but the reserved space for the element is still preserved in the normal flow.

Example

```
h2.pos_top {  
  position: relative;  
  top: -50px;  
}
```

Relatively positioned elements are often used as container blocks for absolutely positioned elements.

Absolute Positioning

An absolute position element is positioned relative to the first parent element that has a position other than static. If no such element is found, the containing block is <html>:

Example

```
h2 {  
  position: absolute;  
  left: 100px; top: 150px;  
}
```

Absolutely positioned elements are removed from the normal flow. The document and other elements behave like the absolutely positioned element does not exist.

Absolutely positioned elements can overlap other elements.

Overlapping Elements

When elements are positioned outside the normal flow, they can overlap other elements.

The z-index property specifies the stack order of an element (which element should be placed in front of, or behind, the others).

An element can have a positive or negative stack order:

Example

```
img {  
  position: absolute;  
  left: 0px;  
  top: 0px;  
  z-index: -1;  
}
```

CSS3 - Gradients

What is Gradients?

CSS gradient is used to display smooth transition within two or more specified colors.

Why CSS Gradient

These are the following reasons to use CSS gradient.

- You don't have to use images to display transition effects.
- The download time and bandwidth usage can also be reduced.
- It provides better look to the element when zoomed, because the gradient is generated by the browser.

Types of gradients

There are two types of gradient in CSS3.

- Linear Gradients(down/up/left/right/diagonally)
- Radial Gradients

1) CSS Linear Gradient

The CSS3 linear gradient goes up/down/left/right and diagonally. To create a CSS3 linear gradient, you must have to define two or more color stops. The color stops are the colors which are used to create a smooth transition. Starting point and direction can also be added along with the gradient effect.

background: linear-gradient (direction, color-stop1, color-stop2.....);

(i) CSS Linear Gradient: (Top to Bottom)

Top to Bottom Linear Gradient is the default linear gradient. Let's take an example of linear gradient that starts from top. It starts red and transitions to green.

1. <!DOCTYPE html>
2. <html>
3. <head>
4. <style>
5. #grad1 {
6. height: 100px;
7. background: -webkit-linear-gradient(red, green); /* For Safari 5.1 to 6.0 */
8. background: -o-linear-gradient(red, green); /* For Opera 11.1 to 12.0 */
9. background: -moz-linear-gradient(red, green); /* For Firefox 3.6 to 15 */
10. background: linear-gradient(red, green); /* Standard syntax (must be last) */
11. }
12. </style>
13. </head>
14. <body>
15. <h3>Linear Gradient - Top to Bottom</h3>

16. <p>This linear gradient starts at the top. It starts red, transitionin
g to green:</p>
17. <div id="grad1"></div>
18. </body>
19. </html>

(ii) CSS Linear Gradient: (Left to Right)

The following example shows the linear gradient that starts from left and goes to right. It starts red from left side and transitioning to green.

1. <!DOCTYPE html>
2. <html>
3. <head>
4. <style>
5. #grad1 {
6. height: 100px;
7. background: -webkit-linear-
gradient(left, red, green); /* For Safari 5.1 to 6.0 */
8. background: -o-linear-
gradient(right, red, green); /* For Opera 11.1 to 12.0 */
9. background: -moz-linear-
gradient(right, red, green); /* For Firefox 3.6 to 15 */
10. background: linear-
gradient(to right, red , green); /* Standard syntax (must be last) */
11. }
12. </style>
13. </head>
14. <body>
15. <h3>Linear Gradient - Left to Right</h3>
16. <p>This linear gradient starts at the left. It starts red, transitionin
g to green:</p>
17. <div id="grad1"></div>
18. </body>
19. </html>

(iii) CSS Linear Gradient: (Diagonal)

If you specify both the horizontal and vertical starting positions, you can make

a linear gradient diagonal.

- <!DOCTYPE html>
- <html>
- <head>
- <style>
- #grad1 {
- height: 100px;
- background: -webkit-linear-gradient(left top, red , green); /* For Safari 5.1 to 6.0 */
- background: -o-linear-gradient(bottom right, red, green); /* For Opera 11.1 to 12.0 */
- background: -moz-linear-gradient(bottom right, red, green); /* For Firefox 3.6 to 15 */
- background: linear-gradient(to bottom right, red , green); /* Standard syntax (must be last) */
- }
- </style>
- </head>
- <body>
- <h3>Linear Gradient - Diagonal</h3>
- <p>This linear gradient starts at top left. It starts red, transitioning to green:</p>
- <div id="grad1"></div>
- </body>
- </html>

2) CSS Radial Gradient

You must have to define at least two color stops to create a radial gradient. It is defined by its center.

- background: radial-gradient(shape size at position, start-color, ..., last-color);

(i) CSS Radial Gradient: (Evenly Spaced Color Stops)

Evenly spaced color stops is a by default radial gradient. Its by default shape is

eclipse, size is farthest- carner, and position is center.

- <!DOCTYPE html>
- <html>
- <head>
- <style>
- #grad1 {
- height: 150px;
- width: 200px;
- background: -webkit-radial-
- gradient(blue, green, red); /* Safari 5.1 to 6.0 */
- background: -o-radial-
- gradient(blue, green, red); /* For Opera 11.6 to 12.0 */
- background: -moz-radial-
- gradient(blue, green, red); /* For Firefox 3.6 to 15 */
- background: radial-
- gradient(blue, green, red); /* Standard syntax (must be last) */
- }
- </style>
- </head>
- <body>
- <h3>Radial Gradient - Evenly Spaced Color Stops</h3>
- <div id="grad1"></div>
- </body>
- </html>

(ii) Radial Gradient: (Differently Spaced Color Stops)

- <!DOCTYPE html>
- <html>
- <head>
- <style>
- #grad1 {
- height: 150px;
- width: 200px;
- background: -webkit-radial-
- gradient(blue 5%, green 15%, red 60%); /* Safari 5.1 to 6.0 */
- background: -o-radial-
- gradient(blue 5%, green 15%, red 60%); /* For Opera 11.6 to 12.0 */

- background: -moz-radial-gradient(blue 5%, green 15%, red 60%); /* For Firefox 3.6 to 15 */
- background: radial-gradient(blue 5%, green 15%, red 60%); /* Standard syntax (must be last) */
- }
- </style>
- </head>
- <body>
- <h3>Radial Gradient - Differently Spaced Color Stops</h3>
- <div id="grad1"></div>
- </body>
- </html>

CSS3 – Shadow

CSS3 supported to add shadow to text or elements. Shadow property has divided as follows –

- Text shadow
- Box Shadow

Text shadow

CSS3 supported to add shadow effects to text.

```
<html>
<head>
  <style>
    h1 {
      text-shadow: 2px 2px;
    }
    h2 {
      text-shadow: 2px 2px red;
    }
    h3 {
      text-shadow: 2px 2px 5px red;
    }
    h4 {
```

```

    color: white;
    text-shadow: 2px 2px 4px #000000;
  }
  h5 {
    text-shadow: 0 0 3px #FF0000;
  }
  h6 {
    text-shadow: 0 0 3px #FF0000, 0 0 5px #0000FF;
  }
  p {
    color: white;
    text-shadow: 1px 1px 2px black, 0 0 25px blue, 0 0 5px darkblue;
  }
</style>
</head>

<body>
  <h1>Tutorialspoint.com</h1>
  <h2>Tutorialspoint.com</h2>
  <h3>Tutorialspoint.com</h3>
  <h4>Tutorialspoint.com</h4>
  <h5>Tutorialspoint.com</h5>
  <h6>Tutorialspoint.com</h6>
  <p>Tutorialspoint.com</p>
</body>
</html>

```

box shadow

Used to add shadow effects to elements,

```

<html>
<head>
  <style>
    div {
      width: 300px;
      height: 100px;
      padding: 15px;
      background-color: red;

```

```

        box-shadow: 10px 10px;
    }
</style>
</head>

<body>
    <div>This is a div element with a box-shadow</div>
</body>
</html>

```

CSS Transforms

CSS3 supports transform property. This transform property facilitates you to translate, rotate, scale, and skews elements.

Transformation is an effect that is used to change shape, size and position.

There are two type of transformation i.e. 2D and 3D transformation supported in CSS3.

CSS3 - 2d Transforms

2D transforms are used to re-change the element structure as translate, rotate, scale, and skew.

The following table has contained common values which are used in 2D transforms –

Sr.No.	Value & Description
1	matrix(n,n,n,n,n,n) Used to defines matrix transforms with six values
2	translate(x,y) Used to transforms the element along with x-axis and y-axis
3	translateX(n) Used to transforms the element along with x-axis

- 4 **translateY(n)**
Used to transforms the element along with y-axis
- 5 **scale(x,y)**
Used to change the width and height of element
- 6 **scaleX(n)**
Used to change the width of element
- 7 **scaleY(n)**
Used to change the height of element
- 8 **rotate(angle)**
Used to rotate the element based on an angle
- 9 **skewX(angle)**
Used to defines skew transforms along with x axis
- 10 **skewY(angle)**
Used to defines skew transforms along with y axis

The translate() method

The CSS translate() method is used to move an element from its current position according to the given parameters i.e. X-axis and Y-axis.

Let's take an example to translate a

element 50 pixels right, and 100 pixels down from its current position.

- <!DOCTYPE html>
- <html>
- <head><meta http-equiv="Content-Type" content="text/html; charset=windows-1252">
- <style>
- div {
- width: 300px;
- height: 100px;

- background-color: lightgreen;
- border: 1px solid black;
- -ms-transform: translate(100px,80px); /* IE 9 */
- -webkit-transform: translate(100px,80px); /* Safari */
- transform: translate(100px,80px); /* Standard syntax */
- }
- </style>
- </head>
- <body>
- <div>
- This div element is translated 50 pixels right, and 100 pixels down from its current position by using translate () method.
- </div>
- </body>
- </html>

The rotate() method

The CSS rotate() method is used to rotate an element clockwise or anti-clockwise according to the given degree.

Rotate 20 degrees

Box rotation with 20 degrees angle as shown below –

```
<html>
<head>
  <style>
    div {
      width: 300px;
      height: 100px;
      background-color: pink;
      border: 1px solid black;
    }
    div#myDiv {
      /* IE 9 */
      -ms-transform: rotate(20deg);

      /* Safari */
```

```

    -webkit-transform: rotate(20deg);

    /* Standard syntax */
    transform: rotate(20deg);
  }
</style>
</head>

<body>
  <div>
    Tutorials point.com.
  </div>

  <div id = "myDiv">
    Tutorials point.com
  </div>
</body>
</html>

```

Rotate -20 degrees

Box rotation with -20 degrees angle as shown below –

```

<html>
<head>
<style>
  div {
    width: 300px;
    height: 100px;
    background-color: pink;
    border: 1px solid black;
  }
  div#myDiv {
    /* IE 9 */
    -ms-transform: rotate(-20deg);

    /* Safari */
    -webkit-transform: rotate(-20deg);

```

```

    /* Standard syntax */
    transform: rotate(-20deg);
  }
</style>
</head>

<body>
  <div>
    Tutorials point.com.
  </div>

  <div id = "myDiv">
    Tutorials point.com
  </div>
</body>
</html>

```

The scale() method

The CSS scale() method is used to increase or decrease the size of an element according to the given width and height.

Let's take an example to increase the size of an element by increasing its width and height two times.

- <!DOCTYPE html>
- <html>
- <head>
- <style>
- div {
- margin: 150px;
- width: 200px;
- height: 100px;
- background-color: lightpink;
- border: 1px solid black;
- border: 1px solid black;
- -ms-transform: scale(2.5,2); /* IE 9 */
- -webkit-transform: scale(2.5,2); /* Safari */
- transform: scale(2.5,2); /* Standard syntax */

- }
- </style>
- </head>
- <body>
- <div>
- This div element is scaled 2.5 times of its original width, and 2 times of its original height.
- </div>
- </body>
- </html>

The skewX() method

The CSS skewX() method is used to skew an element along the X axis according to the given angle.

Skew X axis

Box rotation with skew x-axis as shown below –

```
<html>
<head>
<style>
  div {
    width: 300px;
    height: 100px;
    background-color: pink;
    border: 1px solid black;
  }
  div#skewDiv {
    /* IE 9 */
    -ms-transform: skewX(20deg);

    /* Safari */
    -webkit-transform: skewX(20deg);

    /* Standard syntax */
    transform: skewX(20deg);
  }

```

```
</style>
</head>

<body>
  <div>
    Tutorials point.com.
  </div>

  <div id = "skewDiv">
    Tutorials point.com
  </div>
</body>
</html>
```

The skewY() method

The CSS skewY() method is used to skew an element along the Y axis according to the given angle.

Skew Y axis

Box rotation with skew y-axis as shown below –

```
<html>
<head>
  <style>
    div {
      width: 300px;
      height: 100px;
      background-color: pink;
      border: 1px solid black;
    }
    div#skewDiv {
      /* IE 9 */
      -ms-transform: skewY(20deg);

      /* Safari */
      -webkit-transform: skewY(20deg);
```

```

    /* Standard syntax */
    transform: skewY(20deg);
  }
</style>
</head>

<body>
  <div>
    Tutorials point.com.
  </div>

  <div id = "skewDiv">
    Tutorials point.com
  </div>
</body>
</html>

```

The skew() method

The CSS skew() method is used to skew an element along with X-axis and Y-axis according to the given angle.

- <!DOCTYPE html>
- <html>
- <head>
- <style>
- div {
- width: 300px;
- height: 100px;
- background-color: lightpink;
- border: 1px solid black;
- }
- div#myDiv {
- -ms-transform: skew(30deg,20deg); /* IE 9 */
- -webkit-transform: skew(30deg,20deg); /* Safari */
- transform: skew(30deg,20deg); /* Standard syntax */
- }
- </style>
- </head>

- <body>
- <div>
- This a normal div element.
- </div>
- <div id="myDiv">
- This div element is skewed 30 degrees along the X-axis, and 20 degrees along the Y-axis.
- </div>
- </body>
- </html>

The matrix() method

The CSS matrix() method combines all the six 2D transform methods altogether. It allows you to rotate, scale, translate, and skew elements.

Syntax:

matrix(scaleX(),skewY(),skewX(),scaleY(),translateX(),translateY())

Matrix transform

Box rotation with Matrix transforms as shown below –

```
<html>
<head>
  <style>
    div {
      width: 300px;
      height: 100px;
      background-color: pink;
      border: 1px solid black;
    }
    div#myDiv1 {
      /* IE 9 */
      -ms-transform: matrix(1, -0.3, 0, 1, 0, 0);

      /* Safari */
      -webkit-transform: matrix(1, -0.3, 0, 1, 0, 0);
```

```

    /* Standard syntax */
    transform: matrix(1, -0.3, 0, 1, 0, 0);
  }
</style>
</head>

<body>
  <div>
    Tutorials point.com.
  </div>

  <div id = "myDiv1">
    Tutorials point.com
  </div>
</body>
</html>

```

Matrix transforms with another direction.

```

<html>
<head>
<style>
  div {
    width: 300px;
    height: 100px;
    background-color: pink;
    border: 1px solid black;
  }
  div#myDiv2 {
    /* IE 9 */
    -ms-transform: matrix(1, 0, 0.5, 1, 150, 0);

    /* Safari */
    -webkit-transform: matrix(1, 0, 0.5, 1, 150, 0);

    /* Standard syntax */
    transform: matrix(1, 0, 0.5, 1, 150, 0);
  }

```

```

</style>
</head>

<body>
  <div>
    Tutorials point.com.
  </div>

  <div id = "myDiv2">
    Tutorials point.com
  </div>
</body>
</html>

```

CSS3 - 3D Transforms

The CSS 3D transforms facilitates you to move an element to X-axis, Y-axis and Z-axis.

Methods of 3D transforms

Below methods are used to call 3D transforms –

Sr.No.	Value & Description
1	matrix3d(n,n,n,n,n,n,n,n,n,n,n,n,n,n,n,n) Used to transforms the element by using 16 values of matrix
2	translate3d(x,y,z) Used to transforms the element by using x-axis,y-axis and z-axis
3	translateX(x) Used to transforms the element by using x-axis
4	translateY(y) Used to transforms the element by using y-axis
5	translateZ(z) Used to transforms the element by using y-axis

- 6 **scaleX(x)**
Used to scale transforms the element by using x-axis
- 7 **scaleY(y)**
Used to scale transforms the element by using y-axis
- 8 **scaleZ(z)**
Used to transforms the element by using z-axis
- 9 **rotateX(angle)**
Used to rotate transforms the element by using x-axis
- 10 **rotateY(angle)**
Used to rotate transforms the element by using y-axis
- 11 **rotateZ(angle)**
Used to rotate transforms the element by using z-axis

The 3D rotateX() method (X-axis rotation)

The CSS 3D rotateX() method is used to rotate an element around its X-axis according to the given degree.

- <!DOCTYPE html>
- <html>
- <head>
- <style>
- div {
- width: 300px;
- height: 100px;
- background-color: lightpink;
- border: 1px solid black;
- }
- div#myDiv {
- -webkit-transform: rotateX(150deg); /* Safari */
- transform: rotateX(150deg); /* Standard syntax */
- }
- </style>

- </head>
- <body>
- <div>
- This is JavaTpoint!
- </div>
- <div id="myDiv">
- This is JavaTpoint!
- </div>
- <p>Note: Internet Explorer 9 (and earlier versions) does not support the rotateX() method.</p>
- </body>
- </html>

The 3D rotateY() method (Y-axis rotation)

The CSS 3D rotateY() method is used to rotate an element around its Y-axis according to the given degree.

- <!DOCTYPE html>
- <html>
- <head>
- <style>
- div {
- width: 300px;
- height: 100px;
- background-color:lightpink;
- border: 1px solid black;
- }
-
- div#myDiv {
- -webkit-transform: rotateY(150deg); /* Safari */
- transform: rotateY(150deg); /* Standard syntax */
- }
- </style>
- </head>
- <body>
- <div>
- Welcome to JavaTpoint!.
- </div>

- `<div id="myDiv">`
- Welcome to JavaTpoint!.
- `</div>`
- `<p><b>Note:</b> Internet Explorer 9 (and earlier versions) does not support the rotateY() method.</p>`
- `</body>`
- `</html>`

The 3D rotateZ() method (Z-axis rotation)

The CSS 3D rotateZ() method is used to rotate an element around its Z-axis according to the given degree.

- `<!DOCTYPE html>`
- `<html>`
- `<head>`
- `<style>`
- `div {`
- `width: 300px;`
- `height: 100px;`
- `background-color: yellow;`
- `border: 1px solid black;`
- `}`
- `div#myDiv {`
- `-webkit-transform: rotateZ(90deg); /* Safari */`
- `transform: rotateZ(90deg); /* Standard syntax */`
- `}`
- `</style>`
- `</head>`
- `<body>`
- `<div>`
- Welcome to JavaTpoint!
- `</div>`
- `<div id="myDiv">`
- Welcome to JavaTpoint!
- `</div>`
- `<p><b>Note:</b> Internet Explorer 9 (and earlier versions) does not support the rotateZ() method.</p>`
- `</body>`

- </html>

CSS Transitions

The CSS transitions are effects that are added to change the element gradually from one style to another, without using flash or [JavaScript](#).

You should specify two things to create [CSS](#) transition.

- The CSS property on which you want to add an effect.
- The time duration of the effect.

There are four CSS properties that you should use, all or in part (at least two, transition-property and transition-duration, is a must), to animate the transition. All these properties must be placed along with other CSS properties of the initial state of the element:

- transition
- transition-delay
- transition-duration
- transition-property
- transition-timing-function

1. transition-property: This property allows you to select the CSS properties which you want to animate during the transition(change).

Syntax:

transition-property: none | all | property | property1, property2, ..., propertyN;

- **Values:**
 - **none** is used to specify that no property should be selected.
 - **all** is used to specify all the properties to be selected, though not all properties are animate-able, only the properties which are animate-able will be influenced.
 - We can specify a single **property** or a set of comma-separated properties **property1, property2, ..., propertyN**.

2.transition-duration: This property allows you to determine how long it will take to complete the transition from one CSS property to the other.

Syntax:

transition-duration: time;

- Here, **time** can be in seconds(s) or milliseconds(ms), you should use 's' or 'ms' after the number (without quotes).

3. transition-timing-function: This property allows you to determine the speed of change and the manner of change, during the transition. Like, the change should be fast at the beginning and slow at the end, etc.

Syntax:

transition-timing-function: ease|ease-in|ease-out|ease-in-out|linear|step-start|step-end;

- Note, there are other values that this transition-timing-function can take, only the most frequent and simple are mentioned here.

4. transition-delay: This property allows you to determine the amount of time to wait before the transition actually starts to take place.

Syntax:

transition-delay: time;

- Here, again, **time** can be in seconds(s) or milliseconds(ms), and you should use 's' or 'ms' after the number (without quotes).

The Shorthand Property You can combine all the four transition properties mentioned above, into one single shorthand property, according to the syntax given below. This saves us from writing long codes and prevents from getting messy. Note the ordering of property, it has significance.

Syntax:

transition: (property name) | (duration) | (timing function) | (delay);

The value is taken by are same as mentioned above. This property must be placed with other CSS properties, if any, of the initial state. You should use at least, **property name** and **duration** to get any animate-able effect. Also, the ordering of the values matters. The first value is of the property name, second for the duration and so on, as listed above. So, if only one number is mentioned, it will be taken up as duration, and not as a delay.

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Transition</title>

  <style>
    h1 {
      color: green;
      text-align: center;
    }

    div.one {
      height: 150px;
      width: 150px;
      border: 1px dashed black;
      margin: 0 auto;
      background: #FFEBEE;
      transition: height 2s, width 2s, background 2s;
    }

    div.one:hover {
      height: 300px;
      width: 300px;
      background: #BBDEFB;
    }
  </style>

</head>

<body>
  <h1>GeeksForGeeks</h1>
```

```
<div class="one">
</div>
```

```
</body>
</html>
```

CSS Box Sizing

- The CSS box-sizing property is used to specify how to calculate the total height and width of an element.
- It controls the size of an element with a specified height and width.
- It allows you to include the padding and border within the total height and width of the element.

Without CSS box-sizing

If we don't include box-sizing property, then the box model works like below as default:

- width + padding + border = actual visible width of an element's box
-
- height + padding + border = actual visible height of an element's box

It means if we create a box of some specific height and width and then add padding and border to it, it will look wider than the actual width.

To deal with this issue, developers need to adjust the values for height and width to give space for border and padding.

- <!DOCTYPE html>
- <html lang="en" dir="ltr">
- <head>
- <style media="screen">
- .div1{
- height: 150px;
- width: 400px;
- border: 10px solid green;
-
- }

- .div2{
- height: 150px;
- width: 400px;
- border: 10px solid yellow;
- padding: 20px; }
- </style>
- </head>
- <body>
- <h1>Without box-sizing property</h1>
-
- <div class="div1">
- I am the first div with no padding.
- </div>
- <div class="div2">
- I am the second div with padding added.
- </div>
- </body>
- </html>

Syntax

Syntax of box-sizing property is given as:

- box-sizing: border-box; // for border box
- box-sizing: content-box; // for content box

Value: CSS box-sizing property has two values, which are border-box and content box. These are given as:

- **content-box:** As per the CSS standard, it is the default value of the box-sizing property. By using this value, the height and width properties will only include the content and do not include margin, padding, or border values. It means if we specify the width of the element as 200 px, then the content will be 200px wide, and the border and padding will be added in the final display by making it wider than 200px. For example, if an element has below size specifications:
{width: 500px;
border: 10px solid black;} then it will display a box of size 510px.
- <!DOCTYPE html>

- <html>
- <head>
- <title>box-sizing Property</title>
- <style>
- div {
- width: 300px;
- height: 150px;
- padding: 20px;
- border: 10px solid black;
- background: #4caf50;
- color: red;
- box-sizing: content-box;}
- </style>
- </head>
- <body style = "text-align: center;">
- <h2>box-sizing: content-box</h2>
-

- <div>
- <p>javaTpoint

- <i>Best Portal to learn technologies</i></p>
- </div>
- </body>
- </html>

border-box: The border-box is a widely used value of the box-sizing property. It informs the browser to adjust the border and padding within the specified height and width of an element. It means if we specify the width of an element as 200 px, then the specified border and padding will be included within 200 px, and the content box will adjust the extra width. For example, if an element has the following size specifications:

if you set box-sizing: border-box; on an element, padding and border are included in the width and height:

- <!DOCTYPE html>
- <html lang="en" dir="ltr">
- <head>
- <style media="screen">

```

• .div1{
•   height: 150px;
•   width: 400px;
•   border: 10px solid green;
•   box-sizing: border-box;
•
• }
• .div2{
•   height: 150px;
•   width: 400px;
•   border: 10px solid yellow;
•   padding: 20px;
•   box-sizing: border-box;
•
• }
• </style>
• </head>
• <body>
•   <h1>With box-sizing property</h1>
•
•   <div class="div1">
•     I am the first div with no padding.
•   </div>
•   <div class="div2">
•     I am the second div with padding added.
•   </div>
• </body>
• </html>

```

Example-2

```

• <!DOCTYPE html>
• <html>
•   <head>
•     <title>box-sizing Property</title>
•     <style>
•       div {
•         width: 300px;
•         height: 150px;

```

- padding: 20px;
- border: 10px solid black;
- background: #4caf50;
- color: red;
- box-sizing: border-box;
- }
- </style>
- </head>
- <body style = "text-align: center;">
- <h2>box-sizing: border-box</h2>
-

- <div>
- <p>javaTpoint

- <i>Best Portal to learn technologies</i></p>
- </div>
-
- </body>
- </html>

CSS Media Queries

Media queries can be used to check many things, such as:

- width and height of the viewport
- width and height of the device
- orientation (is the tablet/phone in landscape or portrait mode?)
- resolution

Media queries are for different style rules for different size devices such as mobiles, desktops, etc.

Using media queries are a popular technique for delivering a tailored style sheet to desktops, laptops, tablets, and mobile phones (such as iPhone and Android phones).

Media Query Syntax

A media query consists of a media type and can contain one or more

expressions, which resolve to either true or false.

```
@media not|only mediatype and (expressions) {
  CSS-Code;
}
```

The result of the query is true if the specified media type matches the type of device the document is being displayed on and all expressions in the media query are true. When a media query is true, the corresponding style sheet or style rules are applied, following the normal cascading rules.

Unless you use the not or only operators, the media type is optional and the all type will be implied.

You can also have different stylesheets for different media:

```
<link rel="stylesheet" media="mediatype and|not|only (expressions)"
href="print.css">
```

CSS3 Media Types

Value	Description
All	Used for all media type devices
Print	Used for printers
Screen	Used for computer screens, tablets, smart-phones etc.
Speech	Used for screenreaders that "reads" the page out loud

Examples

One way to use media queries is to have an alternate CSS section right inside your style sheet.

The following example changes the background-color to lightgreen if the viewport is 480 pixels wide or wider (if the viewport is less than 480 pixels, the background-color will be pink):

```
body {
  background-color: pink;
}
```

```
@media screen and (min-width: 480px) {  
  body {  
    background-color: lightgreen;  
  }  
}
```

The following example shows a menu that will float to the left of the page if the viewport is 480 pixels wide or wider (if the viewport is less than 480 pixels, the menu will be on top of the content):

```
@media screen and (min-width: 480px) {  
  #leftsidebar {width: 200px; float: left;}  
  #main {margin-left: 216px;}  
}
```

CSS Flexbox

CSS3 Flexible boxes also known as CSS Flexbox, is a new layout mode in CSS3.

The CSS3 flexbox is used to make the elements behave predictably when they are used with different screen sizes and different display devices. It provides a more efficient way to layout, align and distribute space among items in the container.

CSS Flexbox Layout Module

Before the Flexbox Layout module, there were four layout modes:

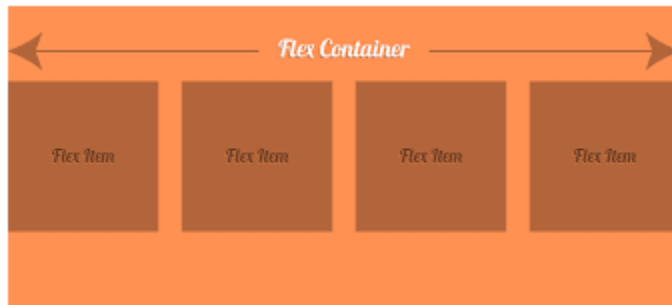
- Block, for sections in a webpage
- Inline, for text
- Table, for two-dimensional table data
- Positioned, for explicit position of an element

The Flexible Box Layout Module, makes it easier to design flexible responsive layout structure without using float or positioning.

The CSS3 flexbox contains flex containers and flex items.

Flex container: The flex container specifies the properties of the parent. It is declared by setting the display property of an element to either flex or inline-flex.

Flex items: The flex items specify properties of the children. There may be one or more flex items inside a flex container.



Note: Flexbox specifies how flex items are set inside a flex container. It sets the flex items inside a flex container along a flex line. By default, there is only one flex line per flex container. Everything outside a flex container and inside a flex item is considered as usual.

Let's take an example to show three flex items within a flex container. By default, they are set along the horizontal flex line, from left to right:

example:

- <!DOCTYPE html>
- <html>
- <head><meta http-equiv="Content-Type" content="text/html; charset=windows-1252">
- <style>
- .flex-container {
- display: -webkit-flex;
- display: flex;
- width: 400px;
- height: 200px;
- background-color: lightpink;
- }
- .flex-item {
- background-color: brown;

- width: 100px;
- height: 100px;
- margin: 10px;
- }
- </style>
- </head>
- <body>
- <div class="flex-container">
- <div class="flex-item">flex item 1</div>
- <div class="flex-item">flex item 2</div>
- <div class="flex-item">flex item 3</div>
- </div>
- </body>
- </html>

You can also change the direction of the flex line by using direction property. If you want to set the direction line right-to-left then set direction property to rtl.

example:

- <!DOCTYPE html>
- <html>
- <head>
- <style>
- body {
- direction: rtl;
- }
- .flex-container {
- display: -webkit-flex;
- display: flex;
- width: 400px;
- height: 200px;
- background-color: lightpink;
- }
- .flex-item {
- background-color: brown;
- width: 100px;
- height: 100px;

- margin: 10px;
- }
- </style>
- </head>
- <body>
- <div class="flex-container">
- <div class="flex-item">flex item 1</div>
- <div class="flex-item">flex item 2</div>
- <div class="flex-item">flex item 3</div>
- </div>
- </body>
- </html>

CSS3 Flexbox Properties

Following is a list of CSS3 Flexbox properties:

property	description
Display	it is used to specify the type of box used for an html element.
flex-direction	it is used to specify the direction of the flexible items inside a flex container.
justify-content	it is used to align the flex items horizontally when the items do not use all available space on the main-axis.
align-items	it is used to align the flex items vertically when the items do not use all available space on the cross-axis.
flex-wrap	it specifies whether the flex items should wrap or not, if there is not enough room for them on one flex line.
align-content	it is used to modify the behavior of the flex-wrap property. it is similar to align-items, but instead of aligning flex items, it aligns flex lines.
flex-flow	it specifies a shorthand property for flex-direction and flex-wrap.
Order	it specifies the order of a flexible item relative to the rest of the flex items inside the same container.
align-self	it is used on flex items. it overrides the container's align-items property.
Flex	it specifies the length of a flex item, relative to the rest of the flex items inside the same container.

CSS Flex Container

Parent Element (Container)

The flex container becomes flexible by setting the display property to flex:

The CSS Flexbox Container Properties

The following table lists all the CSS Flexbox Container properties:

Property	Description
<u>align-content</u>	Modifies the behavior of the flex-wrap property. It is similar to align-items, but instead of aligning flex items, it aligns flex lines
<u>align-items</u>	Vertically aligns the flex items when the items do not use all available space on the cross-axis
<u>display</u>	Specifies the type of box used for an HTML element
<u>flex-direction</u>	Specifies the direction of the flexible items inside a flex container
<u>flex-flow</u>	A shorthand property for flex-direction and flex-wrap
<u>flex-wrap</u>	Specifies whether the flex items should wrap or not, if there is not enough room for them on one flex line
<u>justify-content</u>	Horizontally aligns the flex items when the items do not use all available space on the main-axis

The flex-direction Property

The flex-direction property defines in which direction the container wants to stack the flex items.

Flex-direction:column|column-reverse|row|row-reverse**Example**

The column value stacks the flex items vertically (from top to bottom):

```
.flex-container {  
  display: flex;  
  flex-direction: column;  
}
```

Example

The column-reverse value stacks the flex items vertically (but from bottom to top):

```
.flex-container {  
  display: flex;  
  flex-direction: column-reverse;  
}
```

Example

The row value stacks the flex items horizontally (from left to right):

```
.flex-container {  
  display: flex;  
  flex-direction: row;  
}
```

Example

The row-reverse value stacks the flex items horizontally (but from right to left):

```
.flex-container {  
  display: flex;  
  flex-direction: row-reverse;  
}
```

```
}
```

The flex-wrap Property

The flex-wrap property specifies whether the flex items should wrap or not.

Flex-wrap: wrap|nowrap|wrap-reverse

Example

The wrap value specifies that the flex items will wrap if necessary:

```
.flex-container {  
  display: flex;  
  flex-wrap: wrap;  
}
```

Example

The nowrap value specifies that the flex items will not wrap (this is default):

```
.flex-container {  
  display: flex;  
  flex-wrap: nowrap;  
}
```

Example

The wrap-reverse value specifies that the flexible items will wrap if necessary, in reverse order:

```
.flex-container {  
  display: flex;  
  flex-wrap: wrap-reverse;  
}
```

The flex-flow Property

The flex-flow property is a shorthand property for setting both the flex-

direction and flex-wrap properties.

Example

```
.flex-container {  
  display: flex;  
  flex-flow: row wrap;  
}
```

The justify-content Property

The justify-content property is used to align the flex items:

justify-content:center|flex-start|flex-end|space-around|space-between

Example

The center value aligns the flex items at the center of the container:

```
.flex-container {  
  display: flex;  
  justify-content: center;  
}
```

Example

The flex-start value aligns the flex items at the beginning of the container (this is default):

```
.flex-container {  
  display: flex;  
  justify-content: flex-start;  
}
```

Example

The flex-end value aligns the flex items at the end of the container:

```
.flex-container {
```

```
display: flex;
justify-content: flex-end;
}
```

Example

The space-around value displays the flex items with space before, between, and after the lines:

```
.flex-container {
display: flex;
justify-content: space-around;
}
```

Example

The space-between value displays the flex items with space between the lines:

```
.flex-container {
display: flex;
justify-content: space-between;
}
```

The align-items Property

The align-items property is used to align the flex items.

Align-items:center|flex-start|flex-end|stretch|baseline

Example

The center value aligns the flex items in the middle of the container:

```
.flex-container {
display: flex;
height: 200px;
align-items: center;
}
```

Example

The flex-start value aligns the flex items at the top of the container:

```
.flex-container {  
  display: flex;  
  height: 200px;  
  align-items: flex-start;  
}
```

Example

The flex-end value aligns the flex items at the bottom of the container:

```
.flex-container {  
  display: flex;  
  height: 200px;  
  align-items: flex-end;  
}
```

Example

The stretch value stretches the flex items to fill the container (this is default):

```
.flex-container {  
  display: flex;  
  height: 200px;  
  align-items: stretch;  
}
```

Example

The baseline value aligns the flex items such as their baselines aligns:

```
.flex-container {  
  display: flex;  
  height: 200px;  
  align-items: baseline;  
}
```

```
}
```

The align-content Property

The align-content property is used to align the flex lines.

Example

The space-between value displays the flex lines with equal space between them:

```
.flex-container {  
  display: flex;  
  height: 600px;  
  flex-wrap: wrap;  
  align-content: space-between;  
}
```

Example

Example

The stretch value stretches the flex lines to take up the remaining space (this is default):

```
.flex-container {  
  display: flex;  
  height: 600px;  
  flex-wrap: wrap;  
  align-content: stretch;  
}
```

Example

The center value displays the flex lines in the middle of the container:

```
.flex-container {  
  display: flex;
```

```
height: 600px;  
flex-wrap: wrap;  
align-content: center;  
}
```

Example

The flex-start value displays the flex lines at the start of the container:

```
.flex-container {  
  display: flex;  
  height: 600px;  
  flex-wrap: wrap;  
  align-content: flex-start;  
}
```

Example

The flex-end value displays the flex lines at the end of the container:

```
.flex-container {  
  display: flex;  
  height: 600px;  
  flex-wrap: wrap;  
  align-content: flex-end;  
}
```

Perfect Centering

Set both the justify-content and align-items properties to center, and the flex item will be perfectly centered:

Example

```
.flex-container {  
  display: flex;  
  height: 300px;  
  justify-content: center;  
  align-items: center;  
}
```

```
align-items: center;
}
```

CSS Flex Items

Child Elements (Items)

The direct child elements of a flex container automatically becomes flexible (flex) items.

The CSS Flexbox Items Properties

The following table lists all the CSS Flexbox Items properties:

Property	Description
<u>align-self</u>	Specifies the alignment for a flex item (overrides the flex container's align-items property)
<u>flex</u>	A shorthand property for the flex-grow, flex-shrink, and the flex-basis properties
<u>flex-basis</u>	Specifies the initial length of a flex item
<u>flex-grow</u>	Specifies how much a flex item will grow relative to the rest of the flex items inside the same container
<u>flex-shrink</u>	Specifies how much a flex item will shrink relative to the rest of the flex items inside the same container
<u>order</u>	Specifies the order of the flex items inside the same container

Example

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
```

```
<div>3</div>  
<div>4</div>  
</div>
```

The order Property

The order property specifies the order of the flex items.

The first flex item in the code does not have to appear as the first item in the layout.

The order value must be a number, default value is 0.

Example

The *order* property can change the order of the flex items:

```
<div class="flex-container">  
  <div style="order: 3">1</div>  
  <div style="order: 2">2</div>  
  <div style="order: 4">3</div>  
  <div style="order: 1">4</div>  
</div>
```

The flex-grow Property

The flex-grow property specifies how much a flex item will grow relative to the rest of the flex items.

The value must be a number, default value is 0.

Example

Make the third flex item grow eight times faster than the other flex items:

```
<div class="flex-container">  
  <div style="flex-grow: 1">1</div>  
  <div style="flex-grow: 8">2</div>
```

```
<div style="flex-grow: 8">3</div>  
</div>
```

The flex-shrink Property

The flex-shrink property specifies how much a flex item will shrink relative to the rest of the flex items.

The value must be a number, default value is 1.

Example

Do not let the third flex item shrink as much as the other flex items:

```
<div class="flex-container">  
  <div>1</div>  
  <div>2</div>  
  <div style="flex-shrink: 0">3</div>  
  <div>4</div>  
  <div>5</div>  
  <div>6</div>  
  <div>7</div>  
  <div>8</div>  
  <div>9</div>  
  <div>10</div>  
</div>
```

The flex-basis Property

The flex-basis property specifies the initial length of a flex item.

Example

Set the initial length of the third flex item to 200 pixels:

```
<div class="flex-container">  
  <div>1</div>  
  <div>2</div>
```

```
<div style="flex-basis: 200px">3</div>  
<div>4</div>  
</div>
```

The flex Property

The flex property is a shorthand property for the flex-grow, flex-shrink, and flex-basis properties.

Example

Make the third flex item not growable (0), not shrinkable (0), and with an initial length of 200 pixels:

```
<div class="flex-container">  
  <div>1</div>  
  <div>2</div>  
  <div style="flex: 0 0 200px">3</div>  
  <div>4</div>  
</div>
```

The align-self Property

The align-self property specifies the alignment for the selected item inside the flexible container.

The align-self property overrides the default alignment set by the container's align-items property.

Example

Align the third flex item in the middle of the container:

```
<div class="flex-container">  
  <div>1</div>  
  <div>2</div>  
  <div style="align-self: center">3</div>  
  <div>4</div>
```

</div>

Example

Align the second flex item at the top of the container, and the third flex item at the bottom of the container:

```
<div class="flex-container">
  <div>1</div>
  <div style="align-self: flex-start">2</div>
  <div style="align-self: flex-end">3</div>
  <div>4</div>
</div>
```

CSS Overflow

The **CSS overflow property** *specifies how to handle the content when it overflows* its block level container.

We know that every single element on a page is a rectangular box and the size, positioning and behavior of these boxes are controlled via CSS.

CSS Overflow property values

Value	Description
visible	It specifies that overflow is not clipped. it renders outside the element's box.this is a default value.
hidden	It specifies that the overflow is clipped, and rest of the content will be invisible.
Scroll	It specifies that the overflow is clipped, and a scroll bar is used to see the rest of the content.
Auto	It specifies that if overflow is clipped, a scroll bar is needed to see the rest of the content.

inherit It inherits the property from its parent element.

initial It is used to set the property to its initial value.

CSS Overflow Example

```
<!DOCTYPE html>
<html>
<head>
<style>
div.scroll {
  background-color: #00ffff;
  width: 100px;
  height: 100px;
  overflow: scroll;
}

div.hidden {
  background-color: #00FF00;
  width: 100px;
  height: 170px;
  overflow: hidden;
}
</style>
</head>
<body>
<p>The overflow property specifies what to do if the content of an element
exceeds the size of the element's box.</p>
<p>overflow:scroll</p>
<div class="scroll">You can use the overflow property when you want to h
ave better control of the layout.
The default value is visible.</div>
<p>overflow:hidden</p>
<div class="hidden">You can use the overflow property when you want to
have better control of the layout.
The default value is visible.</div>
</body>
</html>
```

CSS text-overflow

This property specifies the representation of overflowed text, which is not visible to the user. It signals the user about the content that is not visible. This property helps us to decide whether the text should be clipped, show some dots (ellipsis), or display a custom string.

This property does not work on its own. We have to use **white-space: nowrap; and overflow: hidden;** with this property

Syntax

- text-overflow: clip | ellipsis | string | initial | inherit;

Property Values

clip: It is the default value that clips the overflowed text. It truncates the text at the limit of the content area, so that it can truncate the text in the middle of the character.

ellipsis: This value displays an ellipsis (?) or three dots to show the clipped text. It is displayed within the area, decreasing the amount of text.

string: It is used to represent the clipped text to the user using the string of the programmer's choice. It works only in the Firefox browser.

initial: It sets the property to its default value.

inherit: It inherits the property from its parent element.

Example

- <!DOCTYPE html>
- <html>
- <head>
- <style>
- div{
- white-space: nowrap;

- height: 30px;
- width: 250px;
- overflow: hidden;
- border: 2px solid black;
- font-size: 25px;
-
- }
- .jtp{
- text-overflow: clip;
- }
- .jtp1 {
- text-overflow: ellipsis;
- }
-
- h2{
- color: blue;
- }
- div:hover {
- overflow: visible;
- }
- p{
- font-size: 25px;
- font-weight: bold;
- color: red;
- }
- </style>
- </head>
- <center>
- <body>
- <p> Hover over the bordered text to see the full content. </p>
-
- <h2>
- text-overflow: clip;
- </h2>
-
- <div class="jtp">
- Welcome to the javaTpoint.com
- </div>
- <h2>

- text-overflow: ellipsis;
- </h2>
-
- <div class="jtp1">
- Welcome to the javaTpoint.com
- </div>
- </center>
- </body>
- </html>

CSS Cursor

It is used to define the type of mouse cursor when the mouse pointer is on the element. It allows us to specify the cursor type, which will be displayed to the user. When a user hovers on the link, then by default, the cursor transforms into the hand from a pointer.

CSS Syntax

cursor: *value*;

Let's understand the property values of the cursor.

Values	Usage
Alias	It is used to display the indication of the cursor of something that is to be created.
Auto	It is the default property in which the browser sets the cursor.
all-scroll	It indicates the scrolling.
col-resize	Using it, the cursor will represent that the column can be horizontally resized.
Cell	The cursor will represent that a cell or the collection of cells is selected.
context-menu	It indicates the availability of the context menu.
Default	It indicates an arrow, which is the default cursor.
Copy	It is used to indicate that something is copied.

Crosshair	In it, the cursor changes to the crosshair or the plus sign.
e-resize	It represents the east direction and indicates that the edge of the box is to be shifted towards right.
ew-resize	It represents the east/west direction and indicates a bidirectional resize cursor.
n-resize	It represents the north direction that indicates that the edge of the box is to be shifted to up.
ne-resize	It represents the north/east direction and indicates that the edge of the box is to be shifted towards up and right.
Move	It indicates that something is to be shifted.
Help	It is in the form of a question mark or ballon, which represents that help is available.
None	It is used to indicate that no cursor is rendered for the element.
No-drop	It is used to represent that the dragged item cannot be dropped here.
s-resize	It indicates an edge box is to be moved down. It indicates the south direction.
Row-resize	It is used to indicate that the row can be vertically resized.
Se-resize	It represents the south/east direction, which indicates that an edge box is to be moved down and right.
Sw-resize	It represents south/west direction and indicates that an edge of the box is to be shifted towards down and left.
Wait	It represents an hourglass.
<url>	It indicates the source of the cursor image file.
w-resize	It indicates the west direction and represents that the edge of the box is to be shifted left.
Zoom-in	It is used to indicate that something can be zoomed in.
Zoom-out	It is used to indicate that something can be zoomed out.

```

<html>
  <head>
    </head>
    <style>

```

```
body{
  background-color: lightblue;
  color:green;
  text-align: center;
  font-size: 20px;
}
```

```
</style>
```

```
<body>
```

```
<p>Move your mouse over the below words for the cursor change.</p>
```

```
<div style = "cursor:alias">alias Value</div>
<div style = "cursor:auto">auto Value</div>
<div style = "cursor:all-scroll">all-scroll value</div>
<div style = "cursor:col-resize">col-resize value</div>
<div style = "cursor:crosshair">Crosshair</div>
<div style = "cursor:default">Default value</div>
<div style = "cursor:copy">copy value</div>
<div style = "cursor:pointer">Pointer</div>
<div style = "cursor:move">Move</div>
<div style = "cursor:e-resize">e-resize</div>
<div style = "cursor:ew-resize">ew-resize</div>
<div style = "cursor:ne-resize">ne-resize</div>
<div style = "cursor:nw-resize">nw-resize</div>
<div style = "cursor:n-resize">n-resize</div>
<div style = "cursor:se-resize">se-resize</div>
<div style = "cursor:sw-resize">sw-resize</div>
<div style = "cursor:s-resize">s-resize</div>
<div style = "cursor:w-resize">w-resize</div>
<div style = "cursor:text">text</div>
<div style = "cursor:wait">wait</div>
<div style = "cursor:help">help</div>
<div style = "cursor:progress">Progress</div>
<div style = "cursor:no-drop">no-drop</div>
<div style = "cursor:not-allowed">not-allowed</div>
<div style = "cursor:vertical-text">vertical-text</div>
<div style = "cursor:zoom-in">Zoom-in</div>
<div style = "cursor:zoom-out">Zoom-out</div>
```

```
</body>
</html>
```

CSS Visibility

The CSS visibility property is used to specify whether an element is visible or not.

Note: An invisible element also take up the space on the page. By using display property you can create invisible elements that don't take up space.

Syntax:

visibility: visible|hidden|collapse|initial|inherit;

CSS Visibility Parameters

visible:It is the by default value. It specifies that the element is visible.

hidden:It specifies that the element is invisible (but still takes up space).

collapse:It is used only for table elements. It is used to remove a row or column, but it does not affect the table layout.

The space taken up by the row or column will be available for other content.

If collapse is used on other elements, it renders as "hidden"

initial:It is used to set this property to its default value.

inherit:It is used to inherit this property from its parent element.

Example

- <!DOCTYPE html>
- <html>
- <head>
- <style>
- h1.visible {
- visibility: visible

- }
- h1.hidden {
- visibility: hidden
- }
- </style>
- </head>
- <body>
- <h1 class="visible">I am visible</h1>
- <h1 class="hidden">I am invisible</h1>
- <p>Note: An invisible element also take up the space on the page.
- By using display property you can create invisible elements that don't take space.</p>
- </body>
- </html>

CSS filter

CSS filters are used to set visual effects to text, images, and other aspects of a webpage. The CSS **filter** property allows us to access the effects such as color or blur, shifting on the rendering of an element before the element gets displayed.

Syntax

filter: none | invert() | drop-shadow() | brightness() | saturate() | blur() | hue-rotate() | contrast() | opacity() | grayscale() | sepia() | url();

brightness()

As its name implies, it is used to set the brightness of an element. If the brightness is 0%, then it represents completely black, whereas 100% brightness represents the original one. It can also accept values above 100% that provide brighter results.

```
<!DOCTYPE html>
<html>
```

```

<head>
  <title>CSS filter property</title>
  <style>
    body{
      text-align:center;
    }
    #img1 {
      filter: brightness(130%);
    }
  </style>
</head>
<body>
  <img src = "tiger.png" > <h2>Original Image  </h2>
  <img src = "tiger.png" id = "img1"> <h2>brightness(130%)</h2>
</body>

•
• </html>

```

blur()

It is used to apply the blur effect to the element. If the blur value is not specified, then the value 0 is used as a default value. The parameter in blur() property does not accept the percentage values. A larger value of it creates more blur.

Example

```

• <!DOCTYPE html>
• <html>
•
• <head>
•   <title>CSS filter property</title>
•   <style>
•     body{
•       text-align:center;
•     }
•     #img1 {
•       filter: blur(2px);

```

- }
- </style>
- </head>
- <body>
- <h2>Original Image </h2>
- <h2>blur(2px)</h2>
- </body>
-
- </html>

invert()

It is used to invert the samples in the input image. Its 100% value represents completely inverted, and 0% values leave the unchanged input. Negative values are not allowed in it.

Example

- <!DOCTYPE html>
- <html>
-
- <head>
- <title>CSS filter property</title>
- <style>
- body{
- text-align:center;
- }
- #img1 {
- filter: invert(60);
- }
- </style>
- </head>
- <body>
- <h2>Original Image </h2>
- <h2>invert(60)</h2>
- </body>
-
- </html>

saturate()

It sets the saturation of an element. The 0% saturation represents the completely unsaturated element, whereas the 100% saturation represents the original one. The values greater than 100% are allowed that provides super-saturated results. We cannot use negative values with this property.

Example

```

• <!DOCTYPE html>
• <html>
•
• <head>
•   <title>CSS filter property</title>
•   <style>
•     body{
•       text-align:center;
•     }
•     #img1 {
•       filter: saturate(40);
•     }
•   </style>
• </head>
• <body>
•   <img src = "tiger.png" > <h2>Original Image  </h2>
•   <img src = "tiger.png" id = "img1"> <h2>saturate(40)</h2>
• </body>
•
• </html>

```

drop-shadow()

It applies the drop-shadow effect to the input image. The values it accepts are **h-shadow**, **v-shadow**, **blur**, **spread**, and **color**.

Example

```

• <!DOCTYPE html>
• <html>
•

```

- <head>
- <title>CSS filter property</title>
- <style>
- body{
- text-align:center;
- }
- #img1 {
- filter: drop-shadow(10px 20px 30px yellow);
- }
- </style>
- </head>
- <body>
- <h2>Original Image </h2>
- <h2> drop-
- shadow(10px 20px 30px yellow);</h2>
- </body>
-
- </html>

contrast()

It adjusts the contrast of the input. Its 0% value will create a completely black image, whereas the 100% values leave the unchanged input, i.e., represents the original one. Values greater than 100% are allowed that provides results with less contrast.

Example

- <!DOCTYPE html>
- <html>
-
- <head>
- <title>CSS filter property</title>
- <style>
- body{
- text-align:center;
- }
- #img1 {
- filter: contrast(50%);

- }
- </style>
- </head>
- <body>
- <h2>Original Image </h2>
- <h2> contrast(50%)</h2>
- </body>
-
- </html>

opacity()

It is used to apply transparency to the input image. Its 0% value indicates completely transparent, whereas the 100% value represents the original image, i.e., fully opaque.

- Example<!DOCTYPE html>
- <html>
-
- <head>
- <title>CSS filter property</title>
- <style>
- body{
- text-align:center;
- }
- #img1 {
- filter: opacity(40%);
- }
- </style>
- </head>
- <body>
- <h2>Original Image </h2>
- <h2> opacity(40%)</h2>
- </body>
-
- </html>

hue-rotate()

It applies a hue-rotation on the input image. Its perimeter value defines the number of degrees around the color circle; the image will be adjusted. Its default value is 0 degree, which represents the original image. Its maximum value is 360 degrees.

Let's understand it by an illustration.

Example

- `<!DOCTYPE html>`
- `<html>`
-
- `<head>`
- `<title>CSS filter property</title>`
- `<style>`
- `body{`
- `text-align:center;`
- `}`
- `#img1 {`
- `filter: hue-rotate(240deg);`
- `}`
- `</style>`
- `</head>`
- `<body>`
- `<img src = "tiger.png" > <h2>Original Image </h2>`
- `<img src = "tiger.png" id = "img1"> <h2> hue-`
- `rotate(240deg)</h2>`
- `</body>`
-
- `</html>`

grayscale()

It converts the input image into black and white. 0% grayscale represents the original one, whereas 100% represents completely grayscale. It converts the object colors into 256 shades of gray.

Example

- `<!DOCTYPE html>`

- <html>
-
- <head>
- <title>CSS filter property</title>
- <style>
- body{
- text-align:center;
- }
- #img1 {
- filter: grayscale(80%);
- }
- </style>
- </head>
- <body>
- <h2>Original Image </h2>
- <h2> grayscale(80%)</h2>
- </body>
-
- </html>

sepia()

It is used to transform the image into a sepia image. 0% value represents the original image, whereas the 100% value indicates the completely sepia.

Example

- <!DOCTYPE html>
- <html>
-
- <head>
- <title>CSS filter property</title>
- <style>
- body{
- text-align:center;
- }
- #img1 {
- filter: sepia(90%);
- }

- </style>
- </head>
- <body>
- <h2>Original Image </h2>
- <h2> sepia(90%)</h2>
- </body>
-
- </html>

CSS backdrop-filter Property

The backdrop-filter property is used to apply a graphical effect to the area behind an element.

CSS Syntax

backdrop-filter: none|*filter*|initial|inherit;

Property Values

Value	Description
None	Default value. No filter is applied to the backdrop
	A space-separated list of filter-functions like:
<i>Filter</i>	• blur() • brightness() • contrast() • drop-shadow() • grayscale() • hue-rotate() • invert() • opacity() • sepia() • saturate()
	or an url to an SVG filter that will be applied to the backdrop

initial Sets this property to its default value.

inherit Inherits this property from its parent element.

```
<!DOCTYPE html>
<html>
<head>
<style>
div.background {
  background: lightblue url(klematis.jpg) no-repeat center;
  background-size: cover;
  align-items: center;
  display: flex;
  justify-content: center;
  height: 400px;
  width: 400px;
}
```

```
div.transbox {
  background-color: rgba(255, 255, 255, 0.4);
  -webkit-backdrop-filter: blur(5px);
  backdrop-filter: blur(5px);
  padding: 20px;
  margin: 30px;
  font-weight: bold;
}
</style>
</head>
<body>
```

```
<h1>The backdrop-filter Property</h1>
```

```
<div class="background">
  <div class="transbox">
    <p>backdrop-filter: blur(5px)</p>
  </div>
</div>

</body>
```

</html>

CSS The object-fit Property

The CSS object-fit property is used to specify how an or <video> should be resized to fit its container.

This property tells the content to fill the container in a variety of ways; such as "preserve that aspect ratio" or "stretch up and take up as much space as possible".

```
<!DOCTYPE html>
<html>
<head>
<style>
img {
  width:200px;
  height:300px;
}
</style>
</head>
<body>

<h2>Image</h2>


</body>
</html>
```

Here is where the object-fit property comes in. The object-fit property can take one of the following values:

- fill - This is default. The image is resized to fill the given dimension. If necessary, the image will be stretched or squished to fit
- contain - The image keeps its aspect ratio, but is resized to fit within the given dimension

- cover - The image keeps its aspect ratio and fills the given dimension. The image will be clipped to fit
- none - The image is not resized
- scale-down - the image is scaled down to the smallest version of none or contain

Using object-fit: cover;

If we use object-fit: cover; the image keeps its aspect ratio and fills the given dimension. The image will be clipped to fit:

```
<!DOCTYPE html>
<html>
<head>
<style>
img {
  width: 200px;
  height: 300px;
  object-fit: cover;
}
</style>
</head>
<body>

<h2>Using object-fit: cover</h2>



</body>
</html>
```

Using object-fit: contain;

If we use object-fit: contain; the image keeps its aspect ratio, but is resized to fit within the given dimension:

```
img {
  width: 200px;
```

```
height: 300px;  
object-fit: contain;  
}
```

Using object-fit: fill;

If we use object-fit: fill; the image is resized to fill the given dimension. If necessary, the image will be stretched or squished to fit:

```
img {  
width: 200px;  
height: 300px;  
object-fit: fill;  
}
```

Using object-fit: none;

If we use object-fit: none; the image is not resized:

Example

```
img {  
width: 200px;  
height: 300px;  
object-fit: none;  
}
```

Using object-fit: scale-down;

If we use object-fit: scale-down; the image is scaled down to the smallest version of none or contain:

Example

```
img {  
width: 200px;  
height: 300px;  
object-fit: scale-down;  
}
```

How to use Google fonts in CSS?

It is an important aspect of any web design to choose the right font for the webpage. The use of Google fonts is advantageous because we get elegant fonts for our website. The Google fonts API makes it easier and quicker for everyone to use web-fonts. These fonts have been tested on various browser configurations.

Instead of doing any programming, we simply have to add a special stylesheet link to our [HTML](#) document, then refers to the font family of our choice in the [CSS](#) style.

There are two steps to start using the Google font [API](#) that are as follows:

- First, we have to add a stylesheet link for requesting the desired web font.

```
<link rel = "stylesheet" href = "https://fonts.googleapis.com/css?family=Font+Name">
```

- Then we can style an element with the desired web font, either in a stylesheet or by inline style.

```
.css-selector {
font-family: 'Font Name', serif;
}
```

Or,

```
<div style = "font-family: 'Font Name', serif;">Your text</div>
```

Using @import

We can import the font family from the web font service such as Google fonts. It is to be placed in the [<style> tag](#).

```
@import url('https://fonts.googleapis.com/css?family=Lobster');
```

How to specify styles and font-families in a stylesheet URL

To see the list of fonts, click on the following link:

```
https://fonts.google.com/
```

We have to start with the Google fonts API based URL to determine the URL used in our style sheet link.

```
https://fonts.googleapis.com/css
```

Then we require to add the **family=** [URL](#) parameter, using one or more than one family name and styles.

For example, suppose we need to request the [Roboto](#) font, then it can be done as follows:

```
https://fonts.googleapis.com/css?family=Roboto
```

Note: If there is any space in the font family name, replace it with a plus sign (+).

Suppose, we have to request the font, **Open Sans**; then it can be written as follows:

```
https://fonts.googleapis.com/css?family=Open+Sans
```

For requesting multiple font families, the names should be separate from the pipe (|) character. For example -

```
https://fonts.googleapis.com/css?family=Roboto|Inconsolata|Open+Sans
```

Example

1. <html>
- 2.
3. <head>
4. <link href= 'https://fonts.googleapis.com/css?family=Roboto' rel='style

```

    sheet' />
5. <style>
6. h1, p, h2 {
7. font-family: Roboto;
8. }
9. h1, h2{
10.     color: red;
11.     }
12.     p{
13.         color: blue;
14.     }
15. </style>
16. </head>
17.
18.
19.     <body>
20.     <center>
21.     <h1> Hello World </h1>
22.     <h2> font-family: Roboto; </h2>
23.     <p> Hi, Welcome to the javaTpoint.com. This site is developed so t
        hat students may learn computer science related technologies easily. Th
        e javaTpoint.com is always providing an easy and in-
        depth tutorial on various technologies. No one is perfect in this world, a
        nd nothing is eternally best. But we can try to be better. </p>
24.     </center>
25.     </body>
26.
27.     </html>

```

Example - Using @import rule

```

1. <html>
2.
3. <head>
4. <style>
5. @import url('https://fonts.googleapis.com/css?family=Lobster');
6. h1, h2{
7. color: red;
8. }

```

```
9. #div1{
10.     font-family: Lobster;
11.     font-size: 20px;
12. }
13. p{
14.     color: blue;
15. }
16. </style>
17. </head>
18.
19.
20. <body>
21. <center>
22. <h1> Hello World </h1>
23. <div id ="div1">
24. <h2> font-family: Lobster; </h2>
25. <p> Hi, Welcome to the javaTpoint.com. This site is developed so t
    hat students may learn computer science related technologies easily. Th
    e javaTpoint.com is always providing an easy and in-
    depth tutorial on various technologies. No one is perfect in this world, a
    nd nothing is eternally best. But we can try to be better. </p>
26. </div>
27. </center>
28. </body>
29.
30. </html>
```